Java Streams — Employee Management

Employee Model

```
class Employee {

    private int id;
    private String name;
    private int age;
    private String department;
    private double salary;

    public Employee(int id, String name, int age, String department, double salary) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.department = department;
        this.salary = salary;
    }

    public int getId(){ return id; }
    public String getName(){ return name; }
    public int getAge(){ return age; }
    public String getDepartment(){ return department; }
    public double getSalary(){ return salary; }

    public String toString(){
        return id+" "+name+" "+age+" "+department+" "+salary;
    }
}
```

Dataset

```
List<Employee> employees = Arrays.asList(
        new Employee(1,"Surya",25,"IT",60000),
        new Employee(2,"Mahesh",30,"HR",50000),
        new Employee(3,"Kiran",28,"IT",75000),
        new Employee(4,"Anil",35,"Finance",90000),
        new Employee(5,"Suresh",40,"IT",120000),
        new Employee(6,"Ravi",22,"HR",40000),
        new Employee(7,"John",29,"Finance",85000),
        new Employee(8,"Sam",31,"IT",95000)
);
```

Part – A: Core Stream Operations

1.Print All Employees

```
List<Employee> allEmployees =
        employees.stream()
        .toList();
```

2.Extract Employee Names

```
List<String> employeeNames =
        employees.stream()
        .map(Employee::getName)
        .toList();
```

3.Extract Employee Salaries

```
List<Double> employeeSalaries =
        employees.stream()
        .map(Employee::getSalary)
        .toList();
```

4. Filter Employees Salary > 70000

```
List<Employee> highSalaryEmployees =
        employees.stream()
        .filter(e -> e.getSalary() > 70000)
        .toList();
```

5. Filter Employees From IT Department

```
List<Employee> itEmployees =
        employees.stream()
        .filter(e -> e.getDepartment().equals("IT"))
        .toList();
```

6. Total Employee Count

```
long totalEmployees =
        employees.stream()
        .count();
```

7. HR Employee Count

```
long hrEmployees =
        employees.stream()
        .filter(e -> e.getDepartment().equals("HR"))
        .count();
```

8. Highest Salaried Employee (Sorted)

```
Employee highestSalarySorted =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .findFirst()
        .orElse(null);
```

9. Highest Salaried Employee (max)

```
Employee highestSalary =
        employees.stream()
        .max(Comparator.comparing(Employee::getSalary))
        .orElse(null);
```

10. Lowest Salary Employee

```
Employee lowestSalary =
        employees.stream()
        .min(Comparator.comparing(Employee::getSalary))
        .orElse(null);
```

11. Average Salary

```
double averageSalary =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .average()
        .orElse(0);
```

12. Total Salary Using SUM

```
double totalSalary =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .sum();
```

13. Total Salary Using Reduce

```
double totalSalaryReduce =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .reduce(0, Double::sum);
```

14. Sort Employees by Salary

```
List<Employee> salarySorted =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary))
        .toList();
```

15. Sort Employees by Salary Descending

```
List<Employee> salaryDescending =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .toList();
```

16. Distinct Departments

```
List<String> departments =
        employees.stream()
        .map(Employee::getDepartment)
        .distinct()
        .toList();
```

17. Group Employees by Department

```
Map<String,List<Employee>> employeesByDept =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));
```

18. Count Employees Per Department

```
Map<String,Long> departmentEmployeeCount =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
        ));
```

19. Average Salary Per Department

```
Map<String,Double> averageSalaryPerDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)
        ));
```

20. Highest Salary Employee Per Department

```
Map<String,Optional<Employee>> highestPerDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.maxBy(
                        Comparator.comparing(Employee::getSalary)
                )
        ));
```

21. Partition Employees Salary > 70000

```
Map<Boolean,List<Employee>> salaryPartition =
        employees.stream()
        .collect(Collectors.partitioningBy(
                e -> e.getSalary() > 70000
        ));
```

22. Second Highest Salary

```
double secondHighestSalary =
         employees.stream()
         .sorted(Comparator.comparing(Employee::getSalary).reversed())
         .mapToDouble(Employee::getSalary)
         .skip(1)
         .findFirst()
         .orElse(-1);
```

23. Second Highest Salary Employee

```
Employee secondHighestEmployee =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .skip(1)
        .findFirst()
        .orElse(null);
```

24. Department With Highest Average Salary

```
 Map.Entry<String,Double> highestAvgDept =
         employees.stream()
         .collect(Collectors.groupingBy(
                 Employee::getDepartment,
                 Collectors.averagingDouble(Employee::getSalary)
         ))
         .entrySet()
         .stream()
         .max(Map.Entry.comparingByValue())
         .orElse(null);
```

```
Map.Entry<String,Double> highestAvgDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .sorted(Map.Entry.<String,Double>comparingByValue().reversed())
        .findFirst()
        .orElse(null);
```

25. Department With Most Employees

```
Map.Entry<String,Long> departmentWithMostEmployees =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue())
        .orElse(null);
```

Part B – Ranking / Sorting / Filtering Problems

26. Employees With Salary Above Average

```
  double avgSalary =
          employees.stream()
          .mapToDouble(Employee::getSalary)
          .average()
          .orElse(0);

  List<Employee> employeesAboveAverageSalary =
          employees.stream()
          .filter(e -> e.getSalary() > avgSalary)
          .toList();
```

27. Employees With Salary Below Average

```
List<Employee> employeesBelowAverageSalary =
        employees.stream()
        .filter(e -> e.getSalary() < avgSalary)
        .toList();
```

28. Employees Older Than 30

```
List<Employee> employeesOlderThan30 =
        employees.stream()
        .filter(e -> e.getAge() > 30)
        .toList();
```

29. Count Employees Older Than 30

```
long countOlderThan30 =
        employees.stream()
        .filter(e -> e.getAge() > 30)
        .count();
```

30. Youngest Employee

```
Employee youngestEmployee =
         employees.stream()
         .min(Comparator.comparing(Employee::getAge))
         .orElse(null);
```

```
Employee youngestEmployee =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getAge))
        .findFirst()
        .orElse(null);
```

31. Oldest Employee

```
Employee oldestEmployee =
        employees.stream()
        .max(Comparator.comparing(Employee::getAge))
        .orElse(null);
```

32. Employee Names Sorted Alphabetically

```
 List<String> sortedEmployeeNames =
         employees.stream()
         .map(Employee::getName)
         .sorted()
         .toList();
```

33. Sort Employees by Age

```
List<Employee> employeesSortedByAge =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getAge))
        .toList();
```

34. Sort Employees by Age Descending

```
List<Employee> employeesSortedByAgeDesc =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getAge).reversed())
        .toList();
```

35. Sort Employees by Department then Salary

```
 List<Employee> deptThenSalarySorted =
         employees.stream()
         .sorted(
                 Comparator.comparing(Employee::getDepartment)
                 .thenComparing(Employee::getSalary)
         )
         .toList();
```

36. Employees Salary Between 50000 and 90000

```
List<Employee> salaryBetween50And90 =
        employees.stream()
        .filter(e -> e.getSalary() > 50000 && e.getSalary() < 90000)
        .toList();
```

37. Employee Names in Uppercase

```
List<String> employeeNamesUppercase =
        employees.stream()
        .map(e -> e.getName().toUpperCase())
        .toList();
```

38. Map Employee Name → Salary

```
Map<String,Double> nameSalaryMap =
        employees.stream()
        .collect(Collectors.toMap(
                Employee::getName,
                Employee::getSalary
        ));
```

39. Map Employee ID → Employee Object

```
Map<Integer,Employee> idEmployeeMap =
         employees.stream()
         .collect(Collectors.toMap(
                 Employee::getId,
                 Function.identity()
         ));
```

40. Average Employee Age

```
double averageEmployeeAge =
        employees.stream()
        .mapToInt(Employee::getAge)
        .average()
        .orElse(0);
```

41. Group Employees by Age

```
Map<Integer,List<Employee>> employeesByAge =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getAge));
```

42. Group Employees by Department and Count

```
Map<String,Long> departmentEmployeeCount =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
        ));
```

43. Department Wise Total Salary

```
Map<String,Double> deptTotalSalary =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)
        ));
```

44. Group Employees by Department and List Names

```
Map<String,List<String>> deptEmployeeNames =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(
                        Employee::getName,
                        Collectors.toList()
                )
        ));
```

45. Department Wise Average Age

```
Map<String,Double> deptAverageAge =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingInt(Employee::getAge)
        ));
```

46.Partition Employees Age > 30

```
Map<Boolean,List<Employee>> partitionAge30 =
        employees.stream()
        .collect(Collectors.partitioningBy(
                e -> e.getAge() > 30
        ));
```

47. Partition Employees Salary > 80000

```
Map<Boolean,List<Employee>> partitionSalary80k =
        employees.stream()
        .collect(Collectors.partitioningBy(
                e -> e.getSalary() > 80000
        ));
```

48. Top 5 Highest Paid Employees

```
List<Employee> top5HighestPaidEmployees =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .limit(5)
        .toList();
```

49. Bottom 3 Salaried Employees

```
List<Employee> bottom3Employees =
         employees.stream()
         .sorted(Comparator.comparing(Employee::getSalary))
         .limit(3)
         .toList();
```

50. Bottom 3 Salaries Only

```
List<Double> bottom3Salaries =
        employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary))
        .map(Employee::getSalary)
        .limit(3)
        .toList();
```

Part C – Grouping / Collectors / Aggregations

51. Employee Name → Salary Map

```
Map<String, Double> employeeNameSalaryMap =
        employees.stream()
        .collect(Collectors.toMap(
                Employee::getName,
                Employee::getSalary
        ));
```

52. Employee ID → Employee Map

```
Map<Integer, Employee> employeeIdMap =
        employees.stream()
        .collect(Collectors.toMap(
                Employee::getId,
                Function.identity()
        ));
```

53. Group Employees by Department

```
Map<String, List<Employee>> employeesByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));
```

54. Count Employees per Department

```
Map<String, Long> employeeCountByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
        ));
```

55. Average Salary per Department

```
Map<String, Double> averageSalaryByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)
        ));
```

56. Sum of Salaries per Department

```
Map<String, Double> totalSalaryByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)
        ));
```

57. Employees Names per Department

```
Map<String, List<String>> employeeNamesByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(Employee::getName, Collectors.toList())
        ));
```

58. Department → List of Salaries

```
Map<String, List<Double>> departmentSalaries =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(Employee::getSalary, Collectors.toList())
        ));
```

59. Highest Salary Employee per Department

```
Map<String, Optional<Employee>> highestSalaryPerDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.maxBy(
                        Comparator.comparing(Employee::getSalary)
                )
        ));
```

60. Lowest Salary Employee per Department

```
Map<String, Optional<Employee>> lowestSalaryPerDept =
         employees.stream()
         .collect(Collectors.groupingBy(
                 Employee::getDepartment,
                 Collectors.minBy(
                         Comparator.comparing(Employee::getSalary)
                 )
         ));
```

61. Department → Highest Salary Value

```
Map<String, Double> deptHighestSalary =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                        Collectors.maxBy(Comparator.comparing(Employee::getSalary)),
                        opt -> opt.get().getSalary()
                )
        ));
```

62. Group Employees by Age

```
Map<Integer, List<Employee>> employeesByAge =
         employees.stream()
         .collect(Collectors.groupingBy(Employee::getAge));
```

63. Count Employees per Age

```
Map<Integer, Long> employeeCountByAge =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getAge,
                Collectors.counting()
        ));
```

64.Partition Employees Salary > 70000

```
Map<Boolean, List<Employee>> partitionBySalary =
        employees.stream()
        .collect(Collectors.partitioningBy(
                e -> e.getSalary() > 70000
        ));
```

65. Partition Employees Age > 30

```
Map<Boolean, List<Employee>> partitionByAge =
        employees.stream()
        .collect(Collectors.partitioningBy(
                e -> e.getAge() > 30
        ));
```

66. Employee First Letter Frequency

```
Map<Character, Long> nameFirstLetterFrequency =
        employees.stream()
        .collect(Collectors.groupingBy(
                e -> e.getName().charAt(0),
                Collectors.counting()
        ));
```

67. Employees Grouped by First Letter

```
Map<Character, List<Employee>> employeesByFirstLetter =
        employees.stream()
        .collect(Collectors.groupingBy(
                e -> e.getName().charAt(0)
        ));
```

68. Most Common First Letter

```
Map.Entry<Character, Long> mostCommonFirstLetter =
         nameFirstLetterFrequency.entrySet()
         .stream()
         .max(Map.Entry.comparingByValue())
         .orElse(null);
```

69. Salary Summary Statistics

```
DoubleSummaryStatistics salaryStatistics =
        employees.stream()
        .collect(Collectors.summarizingDouble(
                Employee::getSalary
        ));
```

70. Age Summary Statistics

```
IntSummaryStatistics ageStatistics =
         employees.stream()
         .collect(Collectors.summarizingInt(
                 Employee::getAge
         ));
```

71. Department → Employee Count (Sorted)

```
List<Map.Entry<String, Long>> sortedDepartmentCount =
        employeeCountByDepartment.entrySet()
        .stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .toList();
```

72. Department → Total Salary (Sorted)

```
Map<String, Double> totalSalaryByDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)
        ));
```

```
List<Map.Entry<String, Double>> sortedDeptSalary =
        totalSalaryByDepartment.entrySet()
        .stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .toList();
```

73. Department With Highest Salary Sum

```
Map.Entry<String, Double> departmentWithHighestSalary =
        totalSalaryByDepartment.entrySet()
        .stream()
        .max(Map.Entry.comparingByValue())
        .orElse(null);
```

74. Department With Lowest Salary Sum

```
Map.Entry<String, Double> departmentWithLowestSalary =
        totalSalaryByDepartment.entrySet()
        .stream()
        .min(Map.Entry.comparingByValue())
        .orElse(null);
```

75. Department With Most Employees

```
Map.Entry<String, Long> departmentWithMostEmployees =
         employeeCountByDepartment.entrySet()
         .stream()
         .max(Map.Entry.comparingByValue())
         .orElse(null);
```

76. Department With Least Employees

```
Map.Entry<String, Long> departmentWithLeastEmployees =
        employeeCountByDepartment.entrySet()
        .stream()
        .min(Map.Entry.comparingByValue())
        .orElse(null);
```

77. Department → Employee Names Sorted by Salary

```
Map<String, List<String>> deptEmployeeNamesSorted =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                        Collectors.toList(),
                        list -> list.stream()
                                .sorted(Comparator.comparing(Employee::getSalary))
                                .map(Employee::getName)
                                .toList()
                )
        ));
```

78. Top Paid Employee per Department (Flattened)

```
List<Employee> topPaidEmployeesPerDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment))
        .values()
        .stream()
        .map(list -> list.stream()
                .max(Comparator.comparing(Employee::getSalary))
                .get())
        .toList();
```

79. Department → Highest and Lowest Salary Statistics

```
Map<String, DoubleSummaryStatistics> salaryStatsByDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.summarizingDouble(Employee::getSalary)
        ));
```

80. Departments Sorted Alphabetically

```
List<String> sortedDepartments =
        employees.stream()
        .map(Employee::getDepartment)
        .distinct()
        .sorted()
        .toList();
```

Part D – Advanced Stream Analytics

81.Salary Range (Max - Min)

```
double maxSalary =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .max()
        .orElse(0);

double minSalary =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .min()
        .orElse(0);

double salaryRange = maxSalary - minSalary;
```

82. Department With Lowest Average Salary

```
Map.Entry<String, Double> deptLowestAvgSalary =
       employees.stream()
       .collect(Collectors.groupingBy(
               Employee::getDepartment,
               Collectors.averagingDouble(Employee::getSalary)
       ))
       .entrySet()
       .stream()
       .min(Map.Entry.comparingByValue())
       .orElse(null);
```

83. Department → Average Salary Map

```
Map<String, Double> deptAverageSalary =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)
        ));
```

84. Employees With Salary Greater Than Department Average

```
List<Employee> employeesAboveDeptAvg =
        employees.stream()
        .filter(e -> e.getSalary() > deptAverageSalary.get(e.getDepartment()))
        .toList();
```

85. Employees Above Overall Average Salary

```
double overallAvgSalary =
         employees.stream()
         .mapToDouble(Employee::getSalary)
         .average()
         .orElse(0);

List<Employee> employeesAboveOverallAverage =
         employees.stream()
         .filter(e -> e.getSalary() > overallAvgSalary)
         .toList();
```

86.Department → Highest Salary Value Map

```
Map<String, Double> deptHighestSalaryValue =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                        Collectors.maxBy(
                                Comparator.comparing(Employee::getSalary)
                        ),
                        opt -> opt.get().getSalary()
                )
        ));
```

87.Department → Employee Names Sorted By Salary

```
Map<String, List<String>> deptEmployeeNamesSortedBySalary =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                        Collectors.toList(),
                        list -> list.stream()
                                .sorted(Comparator.comparing(Employee::getSalary))
                                .map(Employee::getName)
                                .toList()
                )
        ));
```

88. Employees With Maximum Name Length

```
int maxNameLength =
        employees.stream()
        .mapToInt(e -> e.getName().length())
        .max()
        .orElse(0);

List<Employee> employeesWithMaxNameLength =
        employees.stream()
        .filter(e -> e.getName().length() == maxNameLength)
        .toList();
```

89.Department With Highest Paid Employee

```
String departmentWithHighestPaidEmployee =
        employees.stream()
        .max(Comparator.comparing(Employee::getSalary))
        .map(Employee::getDepartment)
        .orElse(null);
```

90. Employees With Duplicate Names

```
List<String> duplicateEmployeeNames =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getName,
                Collectors.counting()
        ))
        .entrySet()
        .stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .toList();
```

91. Department → Total Salary Sorted Desc

```
List<Map.Entry<String, Double>> departmentSalarySorted =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .toList();
```

92. Top Paid Employee Per Department (Flattened)

```
List<Employee> topPaidPerDepartment =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment))
        .values()
        .stream()
        .map(list -> list.stream()
                .max(Comparator.comparing(Employee::getSalary))
                .get())
        .toList();
```

93. Department With Youngest Average Age

```
String deptYoungestAverageAge =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingInt(Employee::getAge)
        ))
        .entrySet()
        .stream()
        .min(Map.Entry.comparingByValue())
        .get()
        .getKey();
```

94.Department → List of Salaries

```
Map<String, List<Double>> departmentSalaryList =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(Employee::getSalary, Collectors.toList())
        ));
```

95. Employees Earning More Than Their Department Average

```
List<Employee> employeesAboveDeptAverage =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment))
        .values()
        .stream()
        .flatMap(list -> {
            double avg =
                    list.stream()
                    .mapToDouble(Employee::getSalary)
                    .average()
                    .orElse(0);

            return list.stream()
                    .filter(e -> e.getSalary() > avg);
        })
        .toList();
```

96. Employee Count Per Salary Range

```
Map<String, Long> salaryRangeCount =
        employees.stream()
        .collect(Collectors.groupingBy(
                e -> {
                    if(e.getSalary() < 50000) return "LOW";
                    else if(e.getSalary() < 100000) return "MID";
                    else return "HIGH";
                },
                Collectors.counting()
        ));
```

97. Longest Name Employee Per Department

```
Map<String, Employee> longestNameEmployeePerDept =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                        Collectors.maxBy(
                                Comparator.comparing(
                                        e -> e.getName().length()
                                )
                        ),
                        Optional::get
                )
        ));
```

98. Flatten Department → Employee Names

```
List<String> flattenedEmployeeNames =
        employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(Employee::getName, Collectors.toList())
        ))
        .values()
        .stream()
        .flatMap(List::stream)
        .toList();
```

99. Salary Percentage Contribution Per Employee

```
double totalSalarySum =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .sum();

Map<String, Double> salaryContributionPercent =
        employees.stream()
        .collect(Collectors.toMap(
                Employee::getName,
                e -> (e.getSalary() / totalSalarySum) * 100
        ));
```

100. Group Employees By Age Range

```
Map<String, List<Employee>> employeesByAgeRange =
       employees.stream()
       .collect(Collectors.groupingBy(
               e -> {
                   if(e.getAge() < 25) return "Young";
                   else if(e.getAge() < 35) return "Mid";
                   else return "Senior";
               }
       ));
```

101. Employee Closest To Average Salary

```
double avgSalaryValue =
        employees.stream()
        .mapToDouble(Employee::getSalary)
        .average()
        .orElse(0);

Optional<Employee> employeeClosestToAverage =
        employees.stream()
        .min(Comparator.comparing(
                e -> Math.abs(e.getSalary() - avgSalaryValue)
        ));
```